PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL
ESCOLA POLITÉCNICA
46504-04 · Construção de Software · Turma 330 · 2026/2
Prof. Michael da Costa Mora

PROJETO INTEGRADOR · PACOTE 3

PulsePass

Checkpoint do Desenvolvimento

Estado do código, documentação atualizada dos Pacotes 1 e 2 e evidências de teste

Integrantes: Gabriel Kowaleski e Mayra Bordin
Repositório: **https://github.com/K0W4/pulsepass** (release v0.3.1)
Entrega: 23 de novembro de 2026

1. Resumo do checkpoint

O repositório contém a fatia vertical **R1** do PulsePass: do catálogo de eventos até o ingresso validado na portaria, passando por reserva atômica, pagamento idempotente, Outbox e Kafka, agora com a **SPA**, observabilidade, testes de carga, réplicas e teste de resiliência. Tudo sobe com `docker compose up -d --wait` e o script `scripts/smoke.sh` percorre o fluxo completo pelo gateway, autenticando no Keycloak (inclusive o auto-cadastro).

Indicador	Resultado
Testes automatizados	**62**, todos passando: 21 unitários, 25 de integração com PostgreSQL, Redis e Kafka reais (Testcontainers), 3 do simulador, 5 do gateway e 8 do front-end.
Cobertura de linhas do domínio	Inventário 100%, Checkout 100%, Emissão 93% (meta: 70%, verificada no build).
Teste de mutação (PIT)	100% dos mutantes mortos nos três domínios (meta: 60%).
Smoke ponta a ponta	Passa contra a stack real, incluindo auto-cadastro, RBAC, idempotência, ingresso em PDF, validação do QR, trace único e dashboard.
Carga (k6)	Catálogo: p95 = 71 ms a ~1.000 req/s. Reserva: p95 = 7,4 ms a ~200 req/s, 6.000 reservas, 0% de falhas.
Réplicas e resiliência	2 réplicas de cada serviço de negócio com o fluxo completo passando; Emissão derrubada durante uma compra sem perder o ingresso.
Acessibilidade	Lighthouse 100 nas páginas públicas; axe-core sem violações WCAG A/AA em todas as telas.
Histórias de R1 e R2	20 de 21 atendidas; US14 parcial (sem WebSocket).
Requisitos não funcionais	18 atendidos, 4 parciais, 1 pendentes (seção 4).

> **Limites honestos.** As medições de carga foram feitas num laptop, sem latência de rede real; servem para verificar as metas, não como benchmark. A cobertura de 70% mede o pacote `domain` (máquinas de estado e regras puras); a orquestração dos serviços é exercida pelos testes de integração, mas não entra nesse percentual. O axe-core e o Lighthouse foram executados à mão, não na CI. As proteções de branch do GitHub não foram configuradas. A leitura do QR na portaria é por digitação do código.

1.1 Como reproduzir

```
cp .env.example .env
docker compose up -d --build --wait     # SPA em http://localhost:3000
./scripts/smoke.sh
./scripts/replicas.sh                   # 2 réplicas de cada serviço
./scripts/resiliencia.sh                # derruba a Emissão durante uma compra
./scripts/carga.sh                      # k6: catálogo a 1.000 req/s e reserva a 200 req/s
./mvnw verify                           # unitários + integração + formatação + cobertura do
domínio
(cd frontend && npm ci && npm test && npm run build)
docker compose down -v
```

Pré-requisitos: Docker com Compose e Java 21 (o Maven vem pelo wrapper). O `README.md` do repositório traz os detalhes, as portas e os testes que sustentam cada requisito.

2. Estado do backlog (Pacote 1)

Status das histórias da release R1 e das de R2 que foram entregues junto, com a evidência de cada uma. As histórias não citadas individualmente estão no último grupo.

História	Descrição	Status	Evidência
US01	Criar conta com e-mail e senha	Atendido	Auto-cadastro pela tela de registro do Keycloak, com o papel cliente por padrão (`smoke.sh`).
US02	Entrar e receber token de acesso	Atendido	`smoke.sh` e login pela SPA (OIDC com PKCE).
US04	Papel adequado em cada rota (RBAC)	Atendido	401 e 403 testados em todos os serviços (`ConsumidorEApiIT` , `CheckoutIT` , `EmissaoIT` , `RotasTest`) e no smoke; a SPA esconde o que o papel não permite.
US07	Cadastrar evento	Atendido	`ConsumidorEApiIT` , `smoke.sh` e formulário do organizador na SPA.
US08	Criar lotes com preço, quantidade e período	Atendido	`Fixtures` nos testes, `smoke.sh` e formulário na SPA.
US09	Setores e assentos numerados (R2)	Atendido	Setor marcado cria os assentos; formulário na SPA; usado em todos os testes de reserva.
US11	Listar eventos	Atendido	Catálogo público com cache Redis; SPA e `smoke.sh` .
US12	Detalhes, lotes e disponibilidade	Atendido	`GET /eventos/{id}` ; tela de detalhe da SPA.
US14	Mapa de assentos livre, reservado e vendido	Parcial	Mapa na SPA, com alternativa em lista; a disponibilidade é atualizada por *polling* a cada 5 s. Falta o WebSocket (realtime-service).
US15	Reserva atômica por 10 minutos	Atendido	`ReservaConcorrenteIT` : 500 reservas simultâneas, 1 sucesso e 499 conflitos.
US16	Liberar reservas não pagas	Atendido	`ExpiracaoEOutboxIT` .
US17	Cancelar reserva (R2)	Atendido	`ConsumidorEApiIT.clienteCancelaAReserva...` ; botão na tela de checkout.
US19	Revisar pedido com itens e total	Atendido	`GET /pedidos/{id}` ; tela de checkout com cronômetro.
US20	Pagar o pedido	Atendido	`CheckoutIT` , `smoke.sh` e SPA (verificado no navegador).
US21	Clique repetido não cobra duas vezes	Atendido	`CheckoutIT` : 1 chamada ao gateway; 20 chamadas concorrentes, 1 pagamento. A SPA reutiliza a mesma chave nas repetições.
US23	Pagamento recusado e nova tentativa (R2)	Atendido	`CheckoutIT` : recusa volta a RESERVADO e a segunda tentativa paga; a SPA gera chave nova após a recusa.
US24	Ingresso em PDF com QR Code	Atendido	`EmissaoIT` (PDF lido pelo PDFBox), `smoke.sh` e SPA.
US26	Ver todos os meus ingressos	Atendido	`GET /ingressos` e tela "Meus ingressos" com QR.
US28	Operador lê o QR e vê se é válido	Atendido	`EmissaoIT` , `smoke.sh` ; a tela da portaria recebe o código digitado (sem leitura por câmera).
US29	Ingresso usado é recusado (R2)	Atendido	Resposta 409 com o horário da primeira leitura; 50 leituras simultâneas liberam 1.
US31	Total vendido e ocupação por lote	Atendido	`GET /eventos/{id}/vendas` , só para o dono do evento (`ConsumidorEApiIT`); painel na SPA.

História	Descrição	Status	Evidência
US03, US05, US06, US10, US13, US18, US22, US25, US27, US30, US32 a US39	Renovação automática da sessão, cadastro de organizador, auditoria, publicar evento, busca, fila virtual, publicação por Outbox como história própria, e-mail, tempo real, painéis	Pendente	Fora do escopo do checkpoint. US22 está atendida na prática (Outbox em todos os serviços), mas sem história própria no código.

3. Divergências em relação ao Pacote 2

O desenho do Pacote 2 foi seguido, com as diferenças abaixo. Todas estão registradas aqui e, quando afetam contratos, no documento atualizado do Pacote 2 (seção 8).

Seção do Pacote 2	Desenhado	Como ficou
2.3.2 Checkout	`IdempotencyFilter` como filtro servlet	A lógica está em `PagamentoService` (TX1, gateway fora de transação, TX2). Mesmas regras; chamada concorrente com a mesma chave enquanto a primeira roda recebe 409.
2.3.2 Checkout	Falha do gateway	Responde 503; o pagamento falho passa a ter chave própria (`#falha:id`) para a chave do cliente poder ser reutilizada. Recusa responde 402.
2.3.3 Emissão	`PdfStorage` e `NotificacaoWorker`/`EmailSender`	O PDF é gerado sob demanda, sem armazenamento (serviço continua sem estado). E-mail e a tabela `NOTIFICACAO` ficam para o Pacote 4 (US25 é R2).
2.2 Contêineres	SPA	Implementada em TypeScript (Vite) e servida por Nginx, que também faz proxy de `/api` para o gateway. Atualiza a disponibilidade por *polling* de 5 s, e não por WebSocket.
2.2 Contêineres	iam-service, telemetria-service e realtime-service	Não implementados. Os papéis vêm direto do Keycloak (`realm_access.roles`), e cada serviço valida o JWT pelo JWKS. O cadastro de clientes é o auto-cadastro do Keycloak, com o papel cliente por padrão.
2.2 Contêineres	API Gateway com rate limit	O gateway valida JWT e roteia sob o prefixo `/api`; responde 503 quando o serviço de destino está fora do ar. O rate limit não foi implementado.
2.3.1 Inventário	`FilaController` e fila virtual	Não implementados. O Redis é usado só para o cache do catálogo e para o `SeatLock`.
3 Dados	Tabela de Outbox	`outbox_event` guarda `topic`, `msg_key`, o envelope completo em `body` e o `traceparent`. A reserva ganhou `suspensa_ate` (ADR-008) e o lote tem `check` contra sobre-venda. O ingresso guarda `evento_nome` e `descricao` copiados do pedido.
4 Kafka	Criação de tópicos	Um contêiner `kafka-init` cria os tópicos com as partições da seção 4.2 e a criação automática está desligada. Sem isso, o consumidor que subia primeiro fazia o Kafka criar o tópico com 1 partição (defeito encontrado no smoke). Existem os tópicos de reserva, assento, pedido, ingresso e três DLQs; os de IAM e telemetria não foram criados.
4 Kafka	DLQ	Três tentativas com intervalo de 500 ms e depois o tópico `<tópico>.<grupo>.dlq`, na biblioteca `commons`.
1.1 e 5.3 Observabilidade	OpenTelemetry, Prometheus e Grafana no Compose	Implementado: Micrometer Tracing com exportação OTLP para o Collector e o Jaeger; Prometheus com descoberta por DNS (encontra todas as réplicas); Grafana com dashboard provisionado; logs JSON. O `traceparent` é gravado na linha do Outbox e reaplicado pelo relay, para o trace atravessar a publicação assíncrona.
5 Pipeline	Etapas	Workflow `.github/workflows/ci.yml` com lint, testes, front-end, smoke, varredura de segredos, PIT (perfil Maven `mutacao`, no push em develop e à noite) e publicação no GHCR. O Compose não tem Mailpit.

Seção do Pacote 2	Desenhado	Como ficou
1.1 Stack	Versões	Spring Boot 3.5.16, Spring Cloud 2025.0.3, Resilience4j 2.4.0 (uso programático), PDFBox 3.0.8, ZXing 3.5.4, Testcontainers 2.x, Keycloak 26.0, Vite 6, TypeScript 5.9, keycloak-js 26, Jaeger 1.62, Prometheus 2.55, Grafana 11.3.

4. Requisitos não funcionais: estado e evidência

RNF	Requisito	Estado	Evidência
RNF01	Zero sobre-venda	Atendido	`ReservaConcorrenteIT` (assento e lote); restrição `check` no banco como última barreira.
RNF02	Expiração em 10 min + 30 s	Atendido	`ExpiracaoEOutboxIT` (TTL reduzido a 2 s no teste); suspensão limitada a 30 s.
RNF03	Pagamento idempotente	Atendido	`CheckoutIT`.
RNF04	Transactional Outbox	Atendido	`ExpiracaoEOutboxIT`: com o Kafka pausado o evento fica pendente e sai quando ele volta.
RNF05	Consumo idempotente	Atendido	Duplicata de `PedidoPago`, `ReservaCriada` e `ReservaExpirada` testada nos três serviços.
RNF06	Catálogo p95 ≤ 200 ms a 1.000 req/s	Atendido	k6: p95 = 71 ms a ~1.000 req/s por 30 s, 0% de falhas (`docs/evidencias`). Medido num laptop, sem latência de rede real.
RNF07	Reserva p95 ≤ 500 ms a 200/s	Atendido	k6: p95 = 7,4 ms a ~200 req/s por 30 s; 6.000 reservas criadas, 0% de falhas (mesmo ambiente).
RNF08	Ingresso em até 30 s (p95)	Parcial	Sem telemetria-service. Os traces no Jaeger mostram o tempo entre `PedidoPago e IngressoEmitido`, mas o p95 não foi medido; no smoke o ingresso aparece em poucos segundos.
RNF09	Réplicas sem erro funcional	Atendido	`replicas.sh`: 2 réplicas de cada serviço, grupos Kafka com 2 membros, smoke completo e 8 instâncias monitoradas.
RNF10	Timeout, retry e circuit breaker	Atendido	`CheckoutIT`: 3 tentativas com backoff, 503, pedido volta a RESERVADO.
RNF11	Emissão fora não impede a venda	Atendido	`resiliencia.sh`: com a Emissão parada o pagamento é aprovado; ao voltar, ela emite o ingresso retido no Kafka. O gateway responde 503 (testado em `RotasTest`).
RNF12	JWT stateless (15 min, refresh 7 dias)	Parcial	401 sem token testado; tempos configurados no realm, mas a renovação não foi testada.
RNF13	RBAC em todas as rotas	Atendido	Testes por papel nos serviços e no gateway.
RNF14	QR assinado	Atendido	`DominioTest e EmissaoIT`: adulterado ou de outro segredo é rejeitado.
RNF15	LGPD	Parcial	Senhas só no Keycloak; os serviços guardam apenas o identificador do usuário; sem revisão formal de logs.
RNF16	Observabilidade	Atendido	Micrometer Tracing, OpenTelemetry Collector e Jaeger; Prometheus e Grafana com dashboard; logs JSON com o trace id (o correlation-id). Um único trace atravessa gateway, Checkout, Emissão e Inventário, inclusive pelo Outbox e pelo Kafka (`smoke.sh`).
RNF17	Cobertura > 70% e Testcontainers	Atendido	Domínio com 100%, 100% e 93% de linhas; integração com PostgreSQL, Redis e Kafka reais.
RNF18	Lint e arquitetura documentada	Atendido	Spotless (google-java-format) falha o build; C4 e ADRs no Pacote 2.
RNF19	Ambiente com um comando	Atendido	`docker compose up -d --wait` sobe 15 contêineres saudáveis (mais o de inicialização dos tópicos); verificado do zero nesta máquina.
RNF20	CI bloqueia PR vermelho	Parcial	Workflow executa no GitHub: execução 43073919288 no GitHub, com todos os 11 jobs verdes (lint, testes unitários e de integração, front-end, smoke com o Compose, varredura de segredos, PIT e publicação das imagens no GHCR). A proteção das branches (checks obrigatórios) ainda não foi configurada, então o bloqueio de PR não está ativo.

RNF	Requisito	Estado	Evidência
RNF21	Compra em até 5 telas	Atendido	Detalhe, assentos, checkout e meus ingressos (4 telas), percorridas no navegador.
RNF22	Acessibilidade WCAG AA	Atendido	Lighthouse 100 nas páginas públicas (celular e desktop); axe-core sem violações WCAG A/AA em todas as telas, com os três papéis. Mapa de assentos com alternativa em lista. A auditoria é manual (não roda na CI).
RNF23	Fila virtual por ordem de chegada	Pendente	Não implementada.

5. Atualizações do Pacote 1

O Pacote 1 entregue continua válido. As atualizações são estas:

- **Backlog:** o status por história está na seção 2. Nenhuma história foi removida; as de R2 e R3 não citadas seguem como candidatas.
- **Regras esclarecidas durante a construção:** (1) o pedido nasce CRIADO e vira RESERVADO quando a reserva é confirmada pelo evento do Inventário; (2) pagamento recusado ou falha do gateway devolvem o pedido a RESERVADO, para nova tentativa enquanto a reserva estiver ativa; (3) a capacidade de um lote é controlada por contadores (`qtd_reservada` e `qtd_vendida`); (4) a expiração da reserva fica suspensa por até 30 s durante o pagamento.
- **Processo com IA (SDD):** as três funcionalidades críticas têm spec em `/specs` (reserva atômica, pagamento idempotente, ingresso com QR), escritas antes do código. As regras dos agentes estão versionadas em `.claude/CLAUDE.md`. O código foi gerado com apoio do Claude Code e revisado pelo grupo; os testes de integração, o smoke e a verificação da SPA no navegador serviram de contrapeso e acharam defeitos reais, descritos abaixo.
- **Ferramentas efetivamente usadas:** Java 21, Spring Boot 3.5, Maven, Testcontainers, PIT, k6, GitHub Actions, Docker Compose, Keycloak, OpenTelemetry, Prometheus, Grafana, Jaeger, Vite e TypeScript. Figma, GitHub Projects e Structurizr continuam previstos.
- **Front-end (Pacote 1, seção 5):** a SPA foi implementada seguindo a estrutura e o mapa de navegação dos mockups: catálogo, detalhe, mapa de assentos, checkout com contagem regressiva, meus ingressos, painel e cadastro do organizador e leitor da portaria. O status do pedido em tempo real (tela 5) é feito dentro do checkout, por *polling*; a tela de administração (tela 10) e a leitura do QR por câmera ficam para o Pacote 4.

5.1 Defeitos achados pelos testes e corrigidos

- **Chave de idempotência reutilizada após falha do gateway:** o pagamento falho mantinha a chave e a nova tentativa violava a unicidade (achado em `CheckoutIT`).
- **Erros 409, 422 e 404 virando 403:** o redirecionamento para `/error` caía na regra de negação do Spring Security. Os testes com MockMvc não veem isso; o smoke contra o Compose real encontrou.
- **Tópico criado com 1 partição:** corrida de criação de tópicos entre serviços; corrigida com o contêiner `kafka-init` e criação automática desligada.
- **Limite do segredo do QR:** mutante sobrevivente (32 contra 31 caracteres) fechado com teste de fronteira.
- **Papel padrão sem efeito no auto-cadastro:** o Keycloak só importa os papéis compostos pela lista de papéis do realm; quem se cadastrava não recebia o papel cliente. Corrigido e coberto pelo smoke.
- **Gateway devolvia 500 com o serviço fora do ar:** agora responde 503 (teste em `RotasTest`; achado pelo teste de resiliência).
- **Trace partido pelo Outbox:** a publicação assíncrona perdia o contexto; o `traceparent` passou a ser gravado na linha do Outbox e reaplicado pelo relay.
- **Ordem dos assentos e dos ingressos na SPA:** a ordem alfabética (A1, A10, A11, A2) foi trocada por ordem natural, e os ingressos mais novos aparecem primeiro.

6. Atualizações do Pacote 2

O documento do Pacote 2 foi regenerado como `PulsePass_Pacote2_atualizado.pdf`, com a nova seção 8 (atualizações do Pacote 3) e a tabela de divergências da seção 3 deste documento. O arquivo originalmente entregue foi mantido como `PulsePass_Pacote2_entregue.pdf`.

O Pacote 1 original não tem fonte editável neste repositório, por isso ele é mantido como entregue (`PulsePass_Pacote1_entregue.pdf`) e suas atualizações estão na seção 5 deste documento.

7. Próximos passos (Pacote 4)

- `iam-service`, `telemetria-service` e `realtime-service` (WebSocket no lugar do *polling*); medição do p95 do RNF08.
- E-mail com Mailpit; fila virtual (RNF23); tela de administração e leitura de QR por câmera.
- Rate limit no gateway; proteção das branches e revisão cruzada no GitHub; auditoria de acessibilidade automatizada na CI.
- Publicação de eventos (US10) e renovação automática da sessão testada (RNF12).